

STM32 HAL PRACTICAL HANDBOOK

A Beginner-to-Advanced Guide to STM32 HAL Library and Embedded Development

Focused on practical applications: GPIO, PWM, Motors/ESCs, Servos, UART, ADC, I2C, SPI, CAN, DMA, Interrupts, Timers, RTC, Flash and more.

Prepared as a practical reference for STM32 embedded and drone electronics development.

Table of Contents

1. Understanding STM32 HAL
2. STM32 Project Structure and Initialization
3. GPIO: Digital Input and Output
4. Push Buttons and Debouncing
5. PWM Generation
6. LED Brightness Control
7. Servo Control
8. Motor and ESC PWM Control
9. UART / USART Communication
10. UART Interrupts
11. UART DMA
12. ADC
13. Timers and Timer Interrupts
14. I2C
15. SPI
16. CAN
17. DMA
18. NVIC and Interrupts
19. RTC
20. Watchdog
21. Flash Memory and EEPROM Emulation
22. Clock Configuration
23. Error Handling and Debugging
24. Common HAL Macros
25. HAL vs Register-Level Programming
26. Practical Drone-Oriented Architecture
27. Quick Reference Tables

1. Understanding STM32 HAL

HAL means Hardware Abstraction Layer. It is the STM32 software library provided by STMicroelectronics that gives you C functions to configure and control MCU peripherals without directly manipulating every hardware register.

Typical workflow: configure hardware in STM32CubeMX, generate initialization code, then use HAL APIs in your application.

```
HAL_Init();
SystemClock_Config();
MX_GPIO_Init();
MX_USART1_UART_Init();
MX_TIM2_Init();
```

HAL handles the low-level details

For example, instead of manually setting GPIO registers, you can write:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
```

This makes application code easier to read and portable within STM32 families. However, knowing registers remains valuable for debugging, optimization, and advanced peripheral features.

Important rule

Always check the HAL header and reference manual for your exact STM32 family. Function names are similar across STM32 families, but registers, clock trees, flags, and peripheral capabilities can differ.

2. STM32 Project Structure and Initialization

Typical main.c structure

```
int main(void)
{
    HAL_Init();

    SystemClock_Config();

    MX_GPIO_Init();
    MX_USART1_UART_Init();
    MX_TIM2_Init();

    // Start peripherals here

    while (1)
    {
        // Application code
    }
}
```

HAL_Init() initializes the HAL library and the system tick used by functions such as HAL_Delay().

SystemClock_Config() configures the MCU clock. CubeMX-generated MX_*_Init() functions initialize individual peripherals.

Common initialization functions

Function	Purpose
HAL_Init()	Initialize HAL and SysTick
HAL_Delay(ms)	Blocking delay in milliseconds
HAL_GetTick()	Get HAL millisecond tick
HAL_GetUIDw0/1/2()	Read unique device ID words on supported devices

Function	Purpose
HAL_DeInit()	Deinitialize HAL state

3. GPIO: Digital Input and Output

GPIO means General Purpose Input/Output. It is used for LEDs, buttons, enable signals, chip selects, status pins and many other digital signals.

Output

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
```

SET normally means logic HIGH and RESET means logic LOW. The actual voltage depends on the MCU supply and electrical configuration.

Input

```
GPIO_PinState state;

state = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4);

if (state == GPIO_PIN_SET)
{
    // Input is HIGH
}
else
{
    // Input is LOW
}
```

Button with pull-up

With an internal pull-up and a button connected to ground, the input is usually HIGH when released and LOW when pressed. Therefore, a pressed button is detected with GPIO_PIN_RESET.

```
if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4) == GPIO_PIN_RESET)
{
    // Button pressed
}
```

Important configuration

In CubeMX, configure the pin as GPIO_Output, GPIO_Input, GPIO_EXTI, or Alternate Function as appropriate. For peripheral outputs such as PWM, UART, SPI, and I2C, the GPIO must use the correct Alternate Function.

4. Push Buttons and Debouncing

Mechanical buttons bounce. A single physical press can produce many fast HIGH/LOW transitions. A simple software debounce waits a short period and checks the button again.

```
if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4) == GPIO_PIN_RESET)
{
    HAL_Delay(20);

    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4) == GPIO_PIN_RESET)
    {
        // Confirmed press
    }
}
```

For responsive systems, avoid long blocking delays. Use HAL_GetTick(), timers, or EXTI interrupts with a debounce state machine.

Edge vs level

Level detection continuously checks whether the button is currently pressed. **Edge detection** detects a transition, such as released-to-pressed. EXTI interrupts are useful for edge-triggered events.

5. PWM Generation

PWM (Pulse Width Modulation) rapidly switches a digital output. Duty cycle controls the average energy delivered to a load.

For a timer with a 1 MHz counter clock and ARR=999, the timer period is 1000 microseconds, corresponding to 1 kHz.

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
```

If ARR is 999, a compare value around 500 gives approximately 50% duty cycle. The exact behavior at the endpoints depends on timer configuration.

Core PWM calculations

```
TimerCounterClock = TimerClock / (Prescaler + 1)

PWM_Frequency = TimerCounterClock / (ARR + 1)

DutyCycle(%) ≈ Compare / (ARR + 1) × 100
```

The timer clock is not always equal to the system clock. On STM32 parts with APB prescalers, timer clocks can be multiplied relative to the APB peripheral clock. Always verify the clock tree for your MCU.

6. LED Brightness Control

LED brightness can be controlled by PWM. A higher duty cycle generally produces higher perceived brightness, although human vision is nonlinear.

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

while (1)
{
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 250);
    HAL_Delay(1000);

    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
    HAL_Delay(1000);

    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 750);
    HAL_Delay(1000);

    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 999);
    HAL_Delay(1000);
}
```

For smoother visual dimming, use a ramp instead of large steps. For professional lighting, gamma correction can make perceived brightness more linear.

7. Servo Control

Many hobby servos use a pulse repeated approximately every 20 ms (50 Hz). The pulse width determines the target position. Common values are around 1 ms, 1.5 ms, and 2 ms, but the actual range depends on the servo.

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1000); // ~1 ms
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1500); // ~1.5 ms
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 2000); // ~2 ms
```

Angle mapping

```
void Servo_SetAngle(uint8_t angle)
{
    if (angle > 180)
        angle = 180;

    uint16_t pulse = 1000 + ((uint32_t)angle * 1000) / 180;

    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
}
```

This example assumes a timer tick of 1 microsecond and a servo range of 1–2 ms. Adjust the limits for your specific servo.

8. Motor and ESC PWM Control

Many conventional RC ESCs accept a servo-like PWM command. A common control range is approximately 1000–2000 microseconds at 50 Hz, but ESCs vary and may require calibration.

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1000); // Minimum command
HAL_Delay(2000);

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1500); // Mid command
HAL_Delay(2000);

__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 2000); // Maximum command
```

Safety: Remove propellers during testing. Confirm the ESC's input protocol, throttle range, calibration procedure, and failsafe behavior. Never assume that 1000–2000 microseconds is universal.

9. UART / USART Communication

UART is an asynchronous serial communication interface. Common settings include 115200 baud, 8 data bits, no parity, and 1 stop bit (115200 8N1). Connect TX of one device to RX of the other, RX to TX, and share ground.

Transmit

```
uint8_t msg[] = "HELLO\r\n";

HAL_UART_Transmit(
    &huart1,
    msg,
    sizeof(msg) - 1,
    HAL_MAX_DELAY
);
```

Receive

```
uint8_t data;

HAL_UART_Receive(
    &huart1,
    &data,
    1,
    HAL_MAX_DELAY
);
```

The blocking receive waits until the requested bytes arrive or the timeout expires. This is easy to learn but can block the main loop.

String example

```
uint8_t rx_buffer[20];
```

```
HAL_UART_Receive(&huart1, rx_buffer, 5, 1000);

rx_buffer[5] = '\0';
```

Only add a null terminator if the received data is intended to be treated as a C string and the buffer has enough space.

10. UART Interrupts

Interrupt-based UART lets the CPU continue executing other code while the UART peripheral handles reception.

```
uint8_t rx_byte;

HAL_UART_Receive_IT(&huart1, &rx_byte, 1);

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        // Process rx_byte

        HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
    }
}
```

The callback is called when the requested receive operation completes. Restart reception if you want continuous one-byte reception.

Useful callbacks

Callback	Use
HAL_UART_RxCpltCallback()	Receive interrupt completed
HAL_UART_TxCpltCallback()	Transmit interrupt completed
HAL_UART_ErrorCallback()	UART error handling

11. UART DMA

DMA transfers data between peripherals and memory without requiring the CPU to move every byte. It is useful for continuous telemetry, GPS, MAVLink, logging, and high-rate serial streams.

```
HAL_UART_Receive_DMA(&huart1, rx_buffer, sizeof(rx_buffer));

HAL_UART_Transmit_DMA(&huart1, tx_buffer, tx_length);
```

For continuous streams, circular DMA combined with an idle-line detection strategy is often useful. The exact APIs for idle reception depend on STM32 HAL version and MCU family.

Design idea

UART → DMA → RAM buffer → Parser. The application processes complete packets while DMA continues receiving.

12. ADC

ADC converts analog voltage into a digital value. For an N-bit ADC, the raw code normally ranges from 0 to 2^N-1 .

```
uint32_t adc_value;

HAL_ADC_Start(&hadc1);

HAL_ADC_PollForConversion(&hadc1, 100);

adc_value = HAL_ADC_GetValue(&hadc1);
```



```
HAL_ADC_Stop(&hadc1);
```

Convert ADC reading to voltage

```
voltage = (adc_value * VREF) / ADC_MAX;
```

For a 12-bit ADC and 3.3 V reference, ADC_MAX is typically 4095. Actual accuracy depends on reference voltage, ADC configuration, input impedance, calibration, and noise.

Useful ADC APIs

Function	Purpose
HAL_ADC_Start()	Start ADC conversion
HAL_ADC_PollForConversion()	Wait for conversion
HAL_ADC_GetValue()	Read converted value
HAL_ADC_Stop()	Stop conversion
HAL_ADC_Start_IT()	Start interrupt conversion
HAL_ADC_Start_DMA()	Start DMA conversion

13. Timers and Timer Interrupts

Timers generate accurate time bases, PWM, input capture, output compare, and periodic interrupts.

Base timer

```
HAL_TIM_Base_Start(&htim6);
```

Periodic interrupt

```
HAL_TIM_Base_Start_IT(&htim6);

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        // Periodic task
    }
}
```

Keep interrupt callbacks short. Set flags in the ISR and perform longer processing in the main loop when possible.

14. I2C

I2C is a two-wire synchronous bus using SDA and SCL. It supports multiple devices using addresses. Typical sensor applications include MPU6050, EEPROMs, RTCs, and environmental sensors.

```
uint8_t data;

HAL_I2C_Mem_Read(
    &hi2c1,
    DEVICE_ADDR,
    REGISTER_ADDR,
    I2C_MEMADD_SIZE_8BIT,
    &data,
    1,
    HAL_MAX_DELAY
);
```

Many STM32 HAL I2C APIs expect the 7-bit device address shifted left by one bit. Follow the address convention used by your HAL API and datasheet.

Common APIs

Function	Purpose
HAL_I2C_Master_Transmit()	Send bytes to an I2C device
HAL_I2C_Master_Receive()	Receive bytes
HAL_I2C_Mem_Write()	Write device register/memory
HAL_I2C_Mem_Read()	Read device register/memory
HAL_I2C_IsDeviceReady()	Check device availability

15. SPI

SPI is a synchronous serial bus commonly using SCK, MOSI, MISO, and chip select (NSS/CS). It is widely used for radios, displays, flash memories, and sensors.

```
uint8_t tx = 0x55;
uint8_t rx = 0;

HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);

HAL_SPI_TransmitReceive(
    &hspi1,
    &tx,
    &rx,
    1,
    HAL_MAX_DELAY
);

HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);
```

SPI mode depends on CPOL and CPHA. Clock polarity, phase, bit order, and speed must match the peripheral datasheet.

Useful APIs

Function	Purpose
HAL_SPI_Transmit()	Transmit data
HAL_SPI_Receive()	Receive data
HAL_SPI_TransmitReceive()	Full-duplex transfer
HAL_SPI_Transmit_IT()	Interrupt transmit
HAL_SPI_Transmit_DMA()	DMA transmit

16. CAN

CAN is a robust multi-node bus widely used in vehicles and industrial systems. STM32 HAL provides APIs for configuring filters, transmitting frames, and receiving messages.

```
CAN_TxHeaderTypeDef txHeader;
uint32_t txMailbox;
uint8_t txData[8] = {0};

HAL_CAN_Start(&hcan1);

HAL_CAN_AddTxMessage(
    &hcan1,
    &txHeader,
    txData,
    &txMailbox
```

```
);
```

Before using CAN, configure bit timing, mode, filters, GPIO alternate functions, and physical transceiver hardware. CAN bus termination is normally required at the two ends of the bus.

17. DMA

DMA allows data transfers to occur with minimal CPU involvement. It is especially valuable for ADC sampling, UART streams, SPI transfers, and memory operations.

Typical data path: Peripheral → DMA → RAM. The CPU can process other tasks while the transfer occurs.

Common DMA concepts

Concept	Meaning
Normal mode	Transfer completes and stops
Circular mode	Transfer repeats continuously
Half transfer	Useful for processing half a buffer
Transfer complete	Buffer transfer finished
Memory-to-memory	Copy data between memory regions on supported DMA

DMA configuration is usually generated by CubeMX. The exact DMA stream/channel or request mapping depends on the STM32 family.

18. NVIC and Interrupts

NVIC is the Nested Vectored Interrupt Controller. It manages interrupt enabling and priorities.

```
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 5, 0);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

GPIO EXTI callback

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_15)
    {
        // Handle button or external interrupt
    }
}
```

Avoid lengthy operations inside interrupts. Do not normally use long HAL_Delay() calls inside an ISR.

19. RTC

The Real-Time Clock maintains time and date. It can be used for timestamps, scheduling, data logging, and low-power applications.

```
RTC_TimeTypeDef sTime;
RTC_DateTypeDef sDate;

HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);
```

On many STM32 devices, read time before date to properly latch/read the calendar registers. Backup domain and clock source configuration are important for retaining time across resets.

20. Watchdog

A watchdog resets the MCU if software stops servicing it. This helps recover from unexpected firmware hangs.

```
HAL_IWDG_Start(&hiwdg);

while (1)
{
    HAL_IWDG_Refresh(&hiwdg);

    // Application
}
```

Do not blindly refresh the watchdog from a timer interrupt. A better architecture is to refresh it only when critical application tasks have all reported healthy operation.

21. Flash Memory and EEPROM Emulation

STM32 internal Flash stores program code and can also store persistent configuration data if supported by the device and firmware design.

```
HAL_FLASH_Unlock();

/* Erase required sector/page here */
/* Program required data here */

HAL_FLASH_Lock();
```

Flash has erase/program restrictions and limited write endurance. Do not write a new value every loop iteration. Use wear leveling or EEPROM emulation for frequently changing settings.

Typical uses

Store calibration constants, radio configuration, device IDs, failsafe settings, and user preferences.

22. Clock Configuration

Clock configuration determines CPU and peripheral timing. Common sources include HSI, HSE, MSI on supported devices, and PLL-generated clocks.

Clock errors directly affect UART baud rates, timers, PWM frequencies, ADC timing, and communication buses.

Clock checklist

1. Identify oscillator frequency. 2. Configure PLL. 3. Set system clock. 4. Configure AHB/APB prescalers. 5. Verify peripheral clock. 6. Recalculate timer/UART settings.

For timer calculations, do not assume the timer clock equals HCLK. APB prescalers can change the timer clock according to the MCU clock tree.

23. Error Handling and Debugging

HAL status

Return value	Meaning
HAL_OK	Operation successful
HAL_ERROR	Generic error
HAL_BUSY	Peripheral busy
HAL_TIMEOUT	Operation timed out

```
if (HAL_UART_Transmit(&huart1, data, len, 1000) != HAL_OK)
```

```

{
    Error_Handler();
}

```

Common debugging checklist

• Confirm peripheral clock. • Confirm GPIO Alternate Function. • Confirm pin mapping. • Check baud rate/clock. • Check power and ground. • Check signal with an oscilloscope or logic analyzer. • Check DMA/interrupt configuration. • Check HAL state and error codes. • Verify CubeMX-generated MSP initialization.

24. Common HAL Macros

Macro	Purpose
HAL_MAX_DELAY	Wait indefinitely in APIs that support timeout
__HAL_TIM_SET_COMPARE()	Change timer compare value
__HAL_TIM_SET_AUTORELOAD()	Change timer ARR
__HAL_TIM_GET_COUNTER()	Read timer counter
__HAL_GPIO_EXTI_CLEAR_FLAG()	Clear EXTI flag on supported families
__HAL_RCC_GPIOx_CLK_ENABLE()	Enable GPIO peripheral clock
__HAL_RCC_USARTx_CLK_ENABLE()	Enable USART clock
__HAL_LINKDMA()	Link DMA handle to peripheral handle

Macros beginning with `__HAL_` are implementation helpers and can differ by peripheral/family. Use the header files for your MCU as the authoritative source.

25. HAL vs Register-Level Programming

HAL	Register Level
Easy to read	More hardware-specific
Faster development	Maximum control
CubeMX friendly	Requires reference manual knowledge
Good for application code	Good for low-level drivers and optimization
More abstraction overhead	Minimal abstraction overhead

A strong STM32 engineer should understand both. Start with HAL to learn peripherals and application behavior, then study registers to understand what the HAL is doing underneath.

26. Practical Drone-Oriented Architecture

For a drone controller or embedded flight system, a clean architecture separates hardware drivers, communication, control logic, and application state.

```

Application
|
+-- Flight / Motor Control
+-- Failsafe
+-- Telemetry
|
Middleware
|
+-- Packet Parser
+-- Protocol Handling
|

```

```

Drivers
|
+-- UART / SBUS / MAVLink
+-- SPI / Radio
+-- I2C / IMU
+-- ADC / Joystick
+-- PWM / ESC
+-- GPIO
|
STM32 HAL
|
STM32 Hardware

```

For your type of projects, a useful progression is: GPIO → PWM → UART → ADC → Timers/Interrupts → I2C → SPI → DMA → communication protocols → real-time architecture.

27. Quick Reference Tables

GPIO

Task	HAL API
Set pin HIGH/LOW	HAL_GPIO_WritePin()
Read pin	HAL_GPIO_ReadPin()
Toggle pin	HAL_GPIO_TogglePin()
Read state	GPIO_PIN_SET / GPIO_PIN_RESET

PWM / Timer

Task	HAL API
Start PWM	HAL_TIM_PWM_Start()
Stop PWM	HAL_TIM_PWM_Stop()
Change duty/compare	__HAL_TIM_SET_COMPARE()
Start base timer	HAL_TIM_Base_Start()
Start timer interrupt	HAL_TIM_Base_Start_IT()

UART

Task	HAL API
Blocking transmit	HAL_UART_Transmit()
Blocking receive	HAL_UART_Receive()
Interrupt receive	HAL_UART_Receive_IT()
DMA receive	HAL_UART_Receive_DMA()
Transmit interrupt	HAL_UART_Transmit_IT()
Transmit DMA	HAL_UART_Transmit_DMA()

I2C / SPI / ADC

Peripheral	Common APIs
I2C	HAL_I2C_Master_Transmit(), HAL_I2C_Master_Receive(), HAL_I2C_Mem_Read(), HAL_I2C_Mem_Write()
SPI	HAL_SPI_Transmit(), HAL_SPI_Receive(), HAL_SPI_TransmitReceive()

Peripheral	Common APIs
ADC	HAL_ADC_Start(), HAL_ADC_PollForConversion(), HAL_ADC_GetValue(), HAL_ADC_Start_DMA()

Learning Roadmap

1. GPIO output → LED. 2. GPIO input → button. 3. PWM → LED dimming. 4. PWM → servo/ESC. 5. UART → debug messages. 6. UART receive → command parser. 7. ADC → joystick. 8. I2C → MPU6050. 9. SPI → radio module. 10. Interrupts → responsive events. 11. DMA → high-rate data. 12. Build a complete driver architecture.

Final Practical Advice

Do not try to memorize every HAL function. Learn the peripheral concept first, then know where to find the HAL API. For every peripheral, learn four things: CubeMX configuration, initialization function, basic blocking API, and interrupt/DMA API.

For debugging, always reduce the problem to the smallest test: blink one GPIO, generate one PWM signal, send one UART byte, read one ADC channel, read one I2C register, or transmit one SPI byte. Once the basic peripheral works, build the larger application.

Important: This handbook is a practical learning reference, not a replacement for the reference manual and HAL driver documentation for your exact STM32 MCU. Always verify electrical limits, clock configuration, pin alternate functions, and peripheral capabilities for your specific part.